

Traffic Light Project - FPGA

University of Denver

ENCE 2101, Digital Design

Iris Levey

Submission Date: March 16th, 2026

Introduction

A Field Programmable Gate Array (FPGA) is a programmable logic device that consists of a grid of logic gates and interconnections that allow a user to configure them through programming languages to do a variety of different operations without changing any hardware on the device [1].

This project uses an FPGA to simulate one view of a four-way traffic intersection. This intersection consists of one light for cars going straight, one light for cars turning left, one sensor for cars going straight, one sensor for cars turning left, one light for pedestrians, one countdown for pedestrians, and one crosswalk button for pedestrians (Figure 1).

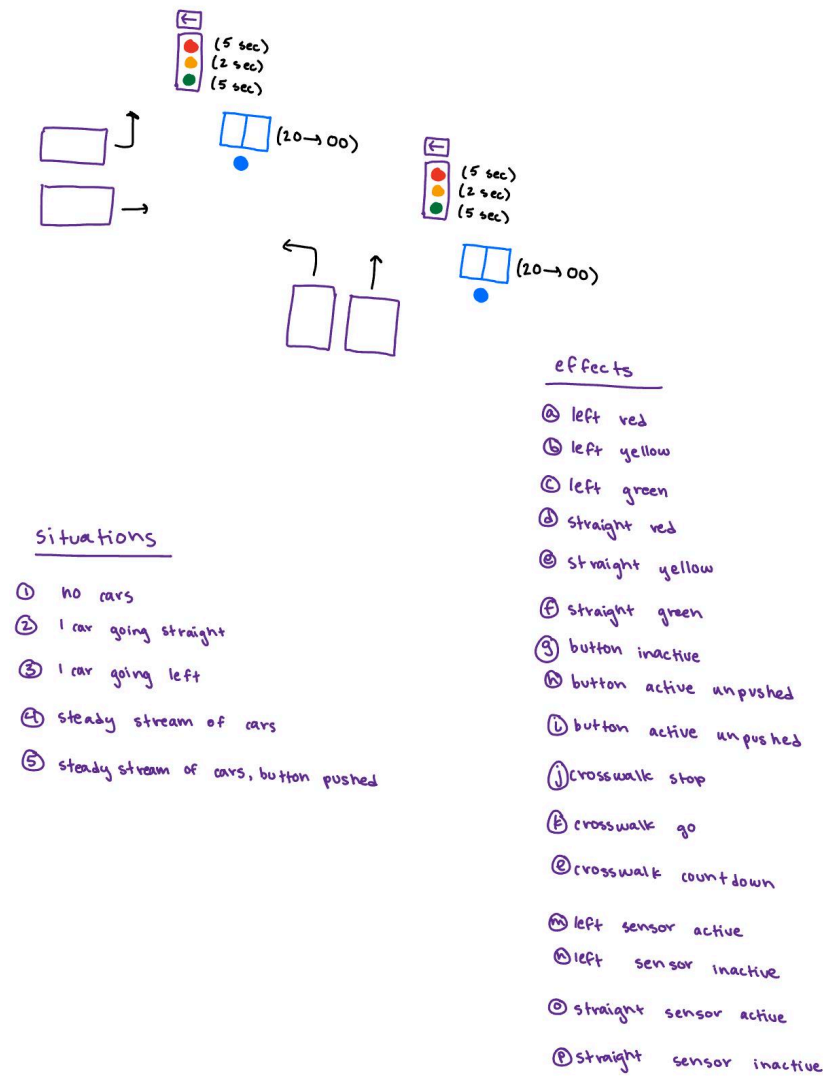
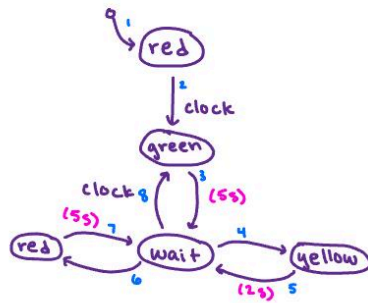


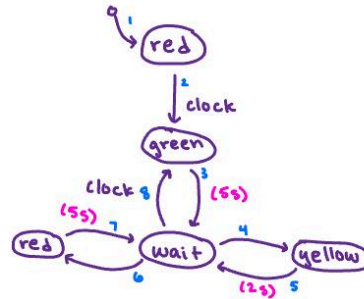
Figure 1. Situation drawing and possible situations and effects

State diagrams representing possible situations and effects within the intersection can be seen in Figure 2.

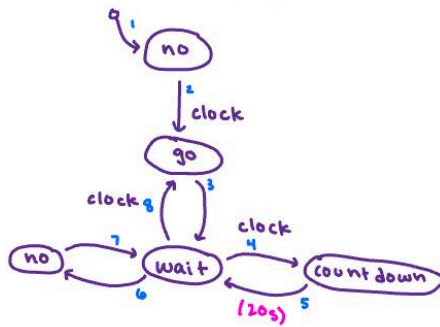
straight light:



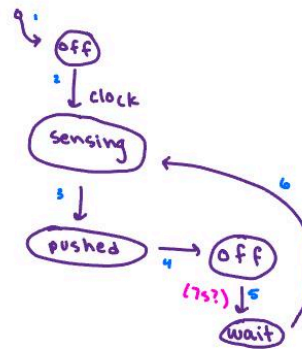
left light:



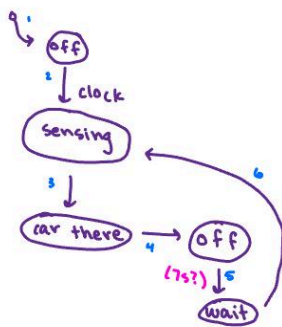
pedestrian light:



pedestrian button



left sensor:



straight sensor:

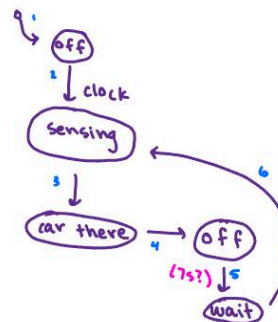


Figure 2. State diagrams

A state diagram of the entire system is shown in Figure 3.

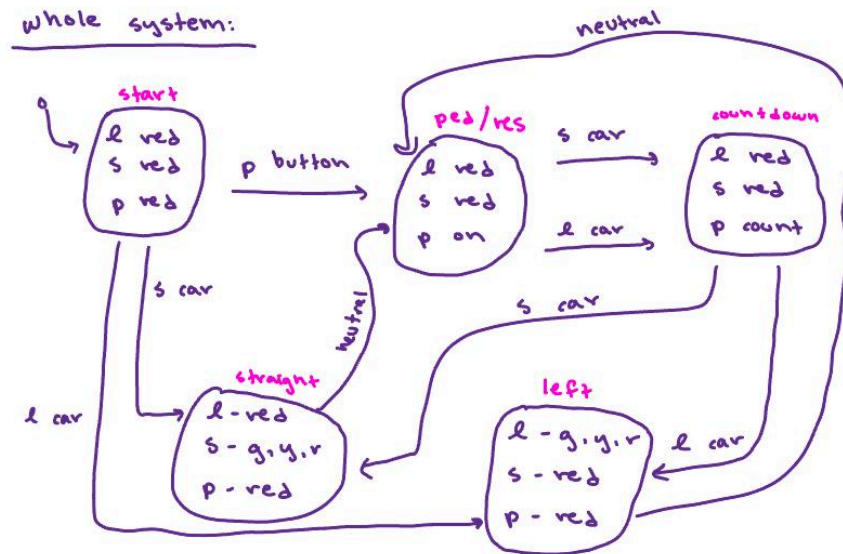


Figure 3. Entire system state diagram

Methods

Apparatus

- Quartus software (Intel® Quartus® Prime Lite Edition Design Software Version 25.1 for Windows) [2]
- 1 computer that runs the Windows operating system
- 1 USB 2.0 A-Male to B-Male cord
- 1 DE10-Lite FPGA
- 1 Prototype Shield V.5
- 6 male to male wires
- 6 female to male wires
- 2 resistors of 330 kΩ resistance
- 2 red LEDs, 2 yellow LEDs, 2 green LEDs

Hardware Setup

All six LEDs and both resistors were inserted into the Prototype Shield V.5's breadboard (Figure 4) which was then placed on top of the DE10-Lite FPGA according to Figure 5.

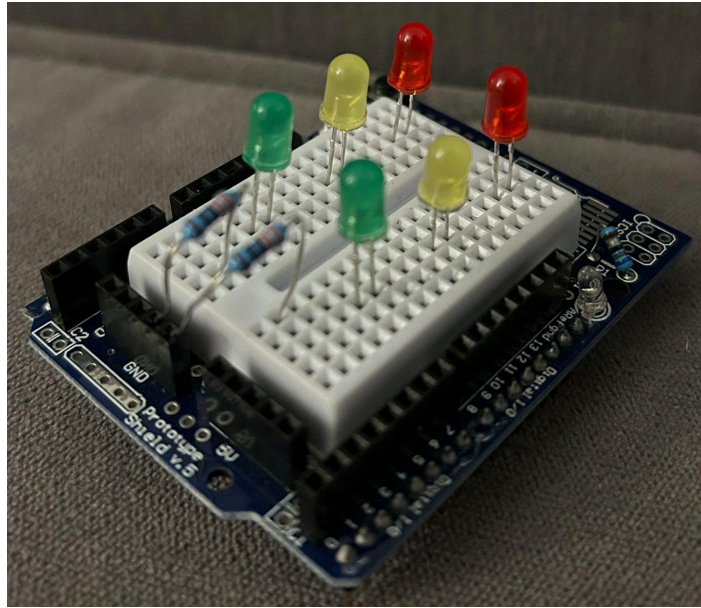
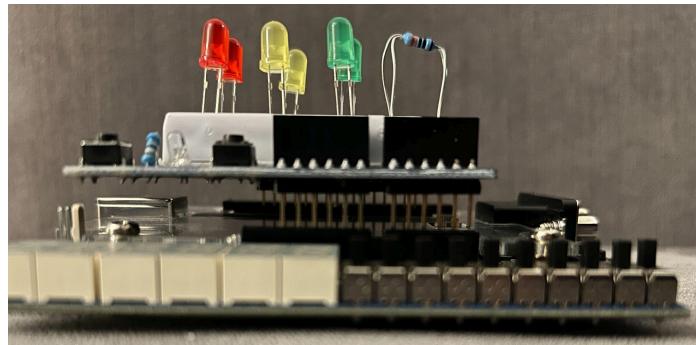


Figure 4. LEDs and resistors to Prototype Shield V.5 connections (positive legs of LED are closer to top of shield)



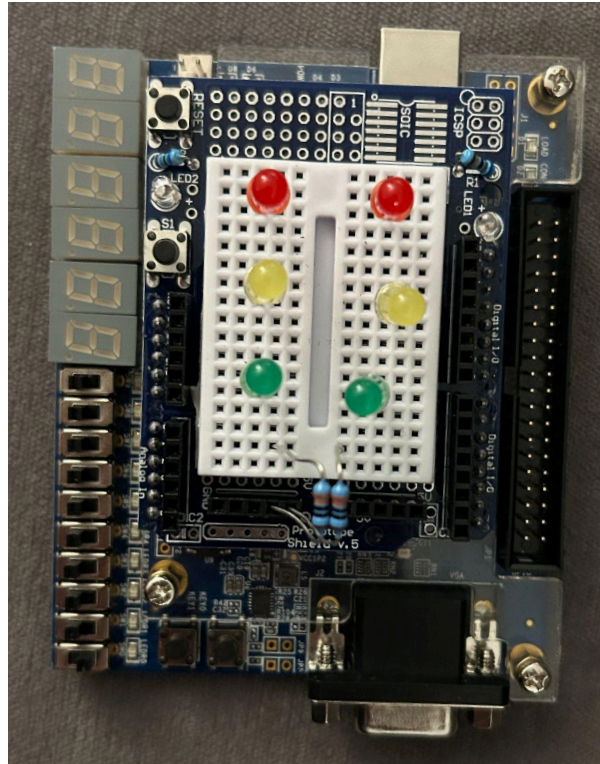
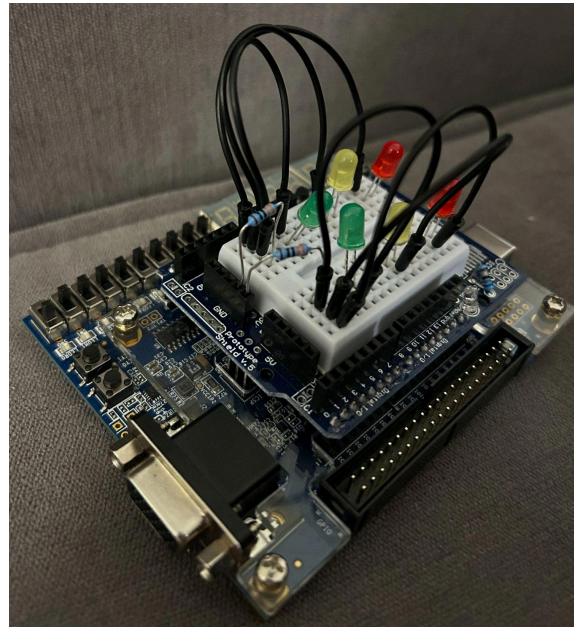
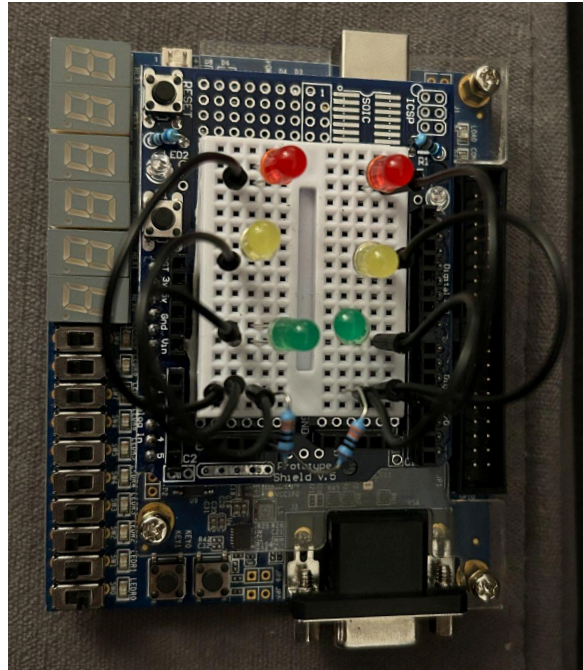


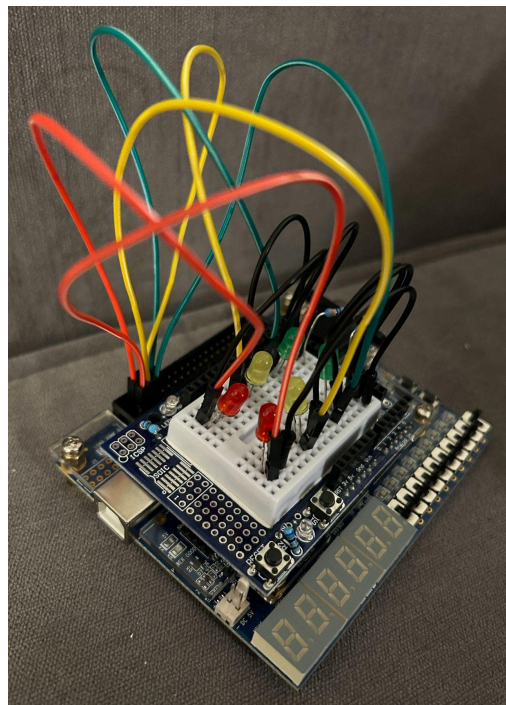
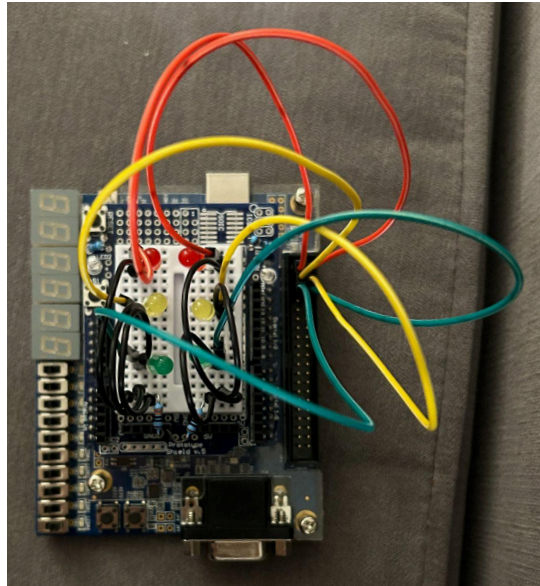
Figure 5. Prototype Shield V.5 to DE10-Lite FPGA connection

The six male to male wires were placed onto the system according to Figure 6, and the six male to female wires were placed according to Figure 7.



- Left red LED negative leg to resistor
- Right red LED negative leg to resistor
- Left yellow LED negative leg to resistor
- Right yellow LED negative leg to resistor
- Left green LED negative leg to resistor
- Right green LED negative leg to resistor

Figure 6. Male to male wire connections



- Left red LED positive leg to GPIO pin 0
- Right red LED positive leg to GPIO pin 1
- Left yellow LED positive leg to GPIO pin 2
- Right yellow LED positive leg to GPIO pin 3
- Left green LED positive leg to GPIO pin 4
- Right green LED positive leg to GPIO pin 5

Figure 7. Male to female wire connections

The A-Male to B-Male cord was used to connect the FPGA to the computer that ran the Windows operating system. All switches on the FPGA were set to low (flipped down).

Software Setup

A new Quartus project was created on the computer. The code shown in Figure 8 was inserted into the main.v file.

```
module main(
    input MAX10_CLK1_50,
    input [9:0] SW,
    input [1:0] KEY,
    output [9:0] LEDR,
    output [6:0] HEX0,
    output [6:0] HEX1,
    output [35:0] GPIO
);

// =====
// 1) DEBOUNCED KEYS
// =====
wire Pushn;
wire Key1;

// 'clean' both of the pushbuttons
debounce db0(
    .clk(MAX10_CLK1_50),
    .key_in(KEY[0]),
    .key_out(Pushn)
);

debounce db1(
    .clk(MAX10_CLK1_50),
    .key_in(KEY[1]),
    .key_out(Key1)
);
```

```

);

//check that key1 was pressed and released
reg key1_prev;

always @(posedge MAX10_CLK1_50) begin
    key1_prev <= Key1; // Key1 is the debounced output
end

wire key1_pressed = (key1_prev == 1 && Key1 == 0); // falling edge

// =====
// 2) SYSTEM START LOGIC (KEY1) + RESET (SW0)
// =====

reg system_started = 0;

always @(posedge MAX10_CLK1_50) begin
    if (!SW[0]) begin
        // SW0 = reset → return to startup state
        system_started <= 0;
    end
    else begin
        // Only allow KEY1 to start the system when not started yet
        if (!system_started && key1_pressed) begin
            system_started <= 1;
        end
    end

end
end

// =====
// 3) REAL 1 Hz CLOCK (for FSM)
// =====
reg [25:0] sec_counter = 0;
reg one_hz_clk = 0;

```

```

always @(posedge MAX10_CLK1_50) begin
    if (sec_counter == 25_000_000) begin
        //reset second counter
        sec_counter <= 0;
        one_hz_clk <= ~one_hz_clk; // toggles every 0.5s → 1 Hz
    end
    else begin
        //increment
        sec_counter <= sec_counter + 1;
    end
end

// =====
// 4) DIRECTION CONTROLLERS (LEFT + STRAIGHT)
// =====

wire left_pre, straight_pre;
wire left_clear, left_enable;
wire straight_clear, straight_enable;
wire left_start, straight_start;
wire left_red_ctrl, left_yel_ctrl, left_grn_ctrl;
wire straight_red_ctrl, straight_yel_ctrl, straight_grn_ctrl;

// LEFT lights controller
direction_controller left_ctrl(
    .clk_1hz(one_hz_clk),
    .reset(SW[0]),
    .switch_up(system_started && SW[3]),
    .straight_pre(straight_pre), //gives straight priority if both are flipped up
    .bcd1(bcd1),
    .bcd0(bcd0),
    .count_done(count_done),
    .fsm_done(l_done),
    .bcd_clear(left_clear),
    .bcd_enable(left_enable),
    .start_fsm(left_start),
    .red_on(left_red_ctrl),
    .yellow_on(left_yel_ctrl),
    .green_on(left_grn_ctrl),

```

```

    .in_pre(left_pre)
);

// STRAIGHT lights controller (priority)
direction_controller straight_ctrl(
    .clk_1hz(one_hz_clk),
    .reset(SW[0]),
    .switch_up(system_started && SW[2]),
    .straight_pre(1'b0), // STRAIGHT has no one above it
    .bcd1(bcd1),
    .bcd0(bcd0),
    .count_done(count_done),
    .fsm_done(s_done),
    .bcd_clear(straight_clear),
    .bcd_enable(straight_enable),
    .start_fsm(straight_start),
    .red_on(straight_red_ctrl),
    .yellow_on(straight_yel_ctrl),
    .green_on(straight_grn_ctrl),
    .in_pre(straight_pre)
);

// =====
// 5) SHARED BCD COUNTER (STRAIGHT PRIORITY)
// =====

//gives straight priority
wire use_straight = straight_pre;
wire use_left    = left_pre && !straight_pre;

//the fsm is running if it's been told that either direction controller is started
wire fsm_running = left_start || straight_start;

//only count down if the fsm isn't running and the count isn't done, otherwise
clear the countdown
wire bcd_enable = show_hex && !count_done && !fsm_running;
wire bcd_clear  = (!show_hex || fsm_running || !SW[0]);

//the countdown is done when 00 is reached

```

```

wire [3:0] bcd1;
wire [3:0] bcd0;
wire count_done = (bcd1 == 4'd0) && (bcd0 == 4'd0);

//one instance of counter (both left and right can use)
bcd_counter shared_bcd(
    .clk(one_hz_clk),
    .clear(bcd_clear),
    .enable(bcd_enable),
    .bcd1(bcd1),
    .bcd0(bcd0)
);

// =====
// 6) HEX DISPLAY (BLANK WHEN NO SWITCH REQUEST)
// =====

//only show hex display when countdown is occuring and not yet done
wire show_hex = (left_pre || straight_pre) && !count_done;

//if there's a countdown show that number, otherwise be blank
wire [3:0] active_bcd1 = show_hex ? bcd1 : 4'hF;
wire [3:0] active_bcd0 = show_hex ? bcd0 : 4'hF;

//create two converters, one for each hex display
bcd_to_7seg seg0_display(
    .bcd(active_bcd0),
    .seg(HEX0)
);

bcd_to_7seg seg1_display(
    .bcd(active_bcd1),
    .seg(HEX1)
);

// =====
// 7) TRAFFIC FSMs (RUN ONLY WHEN CONTROLLER SAYS RUN)

```



```

// =====

//variables for each LED color and to check if the fsms have completed running
for either sensor/light
wire l_red, l_yellow, l_green;
wire s_red, s_yellow, s_green;
wire l_done, s_done;

//fms for the left lights
timer_fsm l_fsm(
    .clk_1hz(one_hz_clk),
    .reset(SW[0]),
    .start(left_start),
    .red(l_red),
    .yellow(l_yellow),
    .green(l_green),
    .done(l_done)
);

//fsm for the straight lights
timer_fsm s_fsm(
    .clk_1hz(one_hz_clk),
    .reset(SW[0]),
    .start(straight_start),
    .red(s_red),
    .yellow(s_yellow),
    .green(s_green),
    .done(s_done)
);

//tell LED 9 to turn on if the system hasn't started yet or if the fsm is running,
otherwise be off
assign LEDR[9] =
    (!system_started) ? 1 : // startup
    (left_start || straight_start) ? 1 : // FSM running
    0; // idle or countdown

// =====
// 8) GPIO OUTPUTS (CONTROLLER RED OR FSM OUTPUTS)
// =====

```

```

// LEFT LEDs assignments for when to be on (red as default)
assign GPIO[0] = left_red_ctrl ? 1 : l_red;
assign GPIO[2] = left_yel_ctrl ? 0 : l_yellow;
assign GPIO[4] = left_grn_ctrl ? 0 : l_green;

// STRAIGHT LEDs assignments for when to be on (red as default)
assign GPIO[1] = straight_red_ctrl ? 1 : s_red;
assign GPIO[3] = straight_yel_ctrl ? 0 : s_yellow;
assign GPIO[5] = straight_grn_ctrl ? 0 : s_green;

endmodule

```

Figure 8. Main code

New Verilog HDL files were created and code inserted into them as well: debounce.v (Figure 9), bcd_counter.v (Figure 10), bcd_to_7seg.v (Figure 11), timer_fsm.v (Figure 12), and direction_controller.v (Figure 13).

```

module debounce(
    input clk,      // 50 MHz clock
    input key_in,   // raw button (active-low)
    output reg key_out // debounced button (still active-low, cleaner)
);

    reg [19:0] cnt = 0; //20 cycles to let clock stabilize
    reg stable = 1; //tracks last known stable value

    always @(posedge clk) begin
        if (key_in != stable) begin //if button is changing
            cnt <= cnt + 1;
            if (cnt == 20'd1_000_000) begin // ~20ms at 50MHz

                //update stored values and reset count
                stable <= key_in;
                key_out <= key_in;
                cnt <= 0;
            end
        end
    end
endmodule

```

```

        end
    end else begin
        cnt <= 0; //input returned to stable value, so ignore this
    end
end
endmodule

```

Figure 9. Debounce code

```

module bcd_counter(
    input clk,
    input clear,
    input enable,
    output reg [3:0] bcd1 = 2,
    output reg [3:0] bcd0 = 0
);
    always @(posedge clk) begin
        //start at 20
        if (clear) begin
            bcd1 <= 2;
            bcd0 <= 0;
        end

        //if at 00 go back to 20
        else if (enable) begin
            if (bcd1 == 0 && bcd0 == 0) begin
                bcd1 <= 2;
                bcd0 <= 0;
            end

            //otherwise count down from 9 to 0 in 1s place
            //increment down by one in the 10s place once 0 is hit
            in 1s place
            else if (bcd0 == 0) begin
                bcd0 <= 9;
                bcd1 <= (bcd1 == 0) ? 9 : bcd1 - 1;
            end
            else
                bcd0 <= bcd0 - 1;
        end
    end
endmodule

```

```

    end
end
endmodule

```

Figure 10. BCD counter code

```

module bcd_to_7seg(
    input [3:0] bcd, //the digit that should be displayed
    output reg [6:0] seg
);
    always @(*) begin
        case (bcd) //set the display to show whichever digit was input
            0: seg = 7'b1000000;
            1: seg = 7'b1111001;
            2: seg = 7'b0100100;
            3: seg = 7'b0110000;
            4: seg = 7'b0011001;
            5: seg = 7'b0010010;
            6: seg = 7'b0000010;
            7: seg = 7'b1111000;
            8: seg = 7'b0000000;
            9: seg = 7'b0010000;

            //turn off all the segments by defaults
            default: seg = 7'b1111111;
        endcase
    end
endmodule

```

Figure 11. BCD to 7 segment display code

```

module timer_fsm(
    input clk_1hz,
    input reset,
    input start,
    output reg red,
    output reg yellow,

```

```

output reg green,
    output reg done
);

//the three states possible
localparam S_RED = 0;
localparam S_GREEN = 1;
localparam S_YEL = 2;

reg [1:0] state = S_RED;
reg [2:0] timer = 0; // enough for 0-5

always @(posedge clk_1hz or negedge reset) begin
    //turn red when reset and when not started - red is default
    if (!reset) begin
        state <= S_RED;
        timer <= 0;
    end else if (!start) begin
        // When not started, hold in RED
        state <= S_RED;
        timer <= 0;
    end
    else begin
        case (state)
            //slight buffer when countdown is done before transitioning
            to green
                S_RED: begin
                    if (timer == 1) begin
                        state <= S_GREEN;
                        timer <= 0;
                    end else begin
                        timer <= timer + 1;
                    end
                end
            //show green for 5 s then go to yellow
            S_GREEN: begin
                if (timer == 5) begin
                    state <= S_YEL;
                    timer <= 0;
                end
            end
        endcase
    end
end

```

```

                                end else begin
                                    timer <= timer + 1;
                                end
                            end
                        //show yellow for 2 seconds then go back to red
S_YEL: begin
    if (timer == 2) begin
        state <= S_RED;
        timer <= 0;
    end else begin
        timer <= timer + 1;
    end
end
    endcase
end

end

//assigning outputs to states
always @(*) begin
    red   = (state == S_RED);
    green = (state == S_GREEN);
    yellow = (state == S_YEL);

    //the lights are done doing their cycle at the final second of yellow
done = (state == S_YEL && timer == 3'd2);
end

endmodule

```

Figure 12. Timer finite state machine code

```

module direction_controller(
    input clk_1hz,
    input reset,
    input switch_up,
    input straight_pre,
    input [3:0] bcd1,

```

```

input [3:0] bcd0,
    input count_done,
    input fsm_done,

output reg bcd_clear,
output reg bcd_enable,
output reg start_fsm,
output reg red_on,
output reg yellow_on,
output reg green_on,
output reg in_pre
);

    //states
localparam IDLE = 0, PRE = 1, RUN = 2;

reg [1:0] state = IDLE;
reg prev_switch = 0;

    //check for pos edge of the clock
wire rising_edge = switch_up && !prev_switch;

always @(posedge clk_1hz or negedge reset) begin

    //if reset immediately go into idle
    if (!reset) begin
        state <= IDLE;
        prev_switch <= 0;
    end

    else begin
        prev_switch <= switch_up;

        case (state)

            IDLE: begin
                // Only start on DOWN→UP edge, and only if not blocked by
STRAIGHT
                if (rising_edge && !straight_pre)
                    state <= PRE;
            end

```

```

PRE: begin //wait for the 20s timer to finish before doing anything
            if (count_done)
                state <= RUN;
            end

RUN: begin
    // in this state until the traffic lights have cycled fully
    if (fsm_done)
        state <= IDLE;
    end

endcase
end
end

always @(*) begin
    in_pre  = (state == PRE);
    start_fsm = (state == RUN);

    // BCD counter only active in PRE
    bcd_clear = (state == PRE);
    bcd_enable = (state == PRE);

    // Red is ON in IDLE and PRE, FSM drives colors in RUN
    red_on  = (state == IDLE || state == PRE);
    yellow_on = 0;
    green_on = 0s;
end

endmodule

```

Figure 13. Direction controller code

The Start Compilation button was pushed to compile the code. Once compiled, the Programmer button was pushed to open a new window and from there, the Start button was pushed to upload the code to the FPGA.

Results

The keys, switches, GPIO pins, LEDs, and HEX displays used are shown in Figure 14.

l:	s:	p:
red - GPIO [4]	red - GPIO [5]	10s - HEX [1]
yellow - GPIO [2]	yellow - GPIO [3]	1s - HEX [0]
green - GPIO [0]	green - GPIO [1]	red - LEDR [9]
sens - SW [3]	sens - SW [2]	button - KEY [1]

Figure 14. Keys, switches, GPIO pins, LEDs, and HEX displays used

Immediately upon receiving the code, both the left and right red LEDs lit up as well as LED 9. Until Switch 0 was flipped up, no matter which keys were pressed or which switches were flipped no other aspects of the FPGA changed. Once Switch 0 was flipped up, pressing Key 1 turned off LED 9 (no other keys or switches besides these ones had any affect on the FPGA).

From there, if either Switch 2 or Switch 3 was flipped up briefly and then back down again, Hex 0 and Hex 1 would display a countdown from 20 to 00 (in the span of 20 seconds). Once it reached 00 the hex displays turned off, LED 9 lit up again, and either the left (if Switch 3 was flipped) or the right (if Switch 2 was flipped) LEDs lit up differently. The green LED was lit up for 5 seconds, the yellow LED for 2 seconds, and the red LED for 5 seconds. LED 9 was then turned off again and the red LED stayed lit.

If at any point during this process Switch 0 was flipped back down, the starting state was returned to with both red LEDs lit up and LED 9 lit up as well.

Discussion

When the system is first turned on, it represents a state where all traffic lights are red as well as the pedestrian crossing light. Switch 0 is flipped up to begin usage of the system/intersection. Pressing Key 1 represents a pedestrian pressing the crosswalk button. This immediately turns off the pedestrian crossing red light, allowing pedestrians to cross the intersection. As soon as a car shows up, though, a 20 second countdown timer begins to tell the pedestrians to exit the intersection. Switches 2 and 3 act as pressure sensors under the road where cars would sit in either the straight lane or left lane respectively. Once the 20 second countdown is finished, the red pedestrian light reappears. Whichever car shows up is given 5 seconds to move before the traffic light returns to red and pedestrians are allowed to walk again. This cycle then repeats indefinitely unless Switch 0 is flipped back down, at which point it returns to its initial state.

References

- [1] SparkFun Electronics, "FPGA - SparkFun Electronics," *SparkFun.com*. [Online]. Available: <https://www.sparkfun.com/fpga> [Accessed: Mar. 15, 2026].
- [2] Intel Corporation, "Intel® Quartus® Prime Lite Edition Design Software Version 25.1 for Windows," *Intel.com*. [Online]. Available: <https://www.intel.com/content/www/us/en/software-kit/868561/intel-quartus-prime-lite-edition-design-software-version-25-1-for-windows.html> [Accessed: Mar. 15, 2026].